

Banker's Algorithm

Deadlock Avoidance & Safety State Verification

The Banker's Algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety by simulating the allocation of predetermined maximum possible amounts of all resources.

System Snapshot Data Structure

To verify if a system is in a safe state, the algorithm requires four matrices: **Allocation** (currently assigned), **Max** (maximum demand), **Available** (free resources), and **Need** ($\text{ext}\{\text{Need}\} = \text{ext}\{\text{Max}\} - \text{ext}\{\text{Allocation}\}$).

```
user@os-sim:~/labs$ cat system_state.json
{
  "Processes": ["P0", "P1", "P2", "P3", "P4"],
  "Resource_Types": ["A", "B", "C"],
  "Total_System_Instances": [10, 5, 7],
  "Available": [3, 3, 2],
  "Allocation": [
    [0, 1, 0],
    // P0

    [2, 0, 0],
    // P1

    [3, 0, 2],
    // P2

    [2, 1, 1],
    // P3

    [0, 0, 2]
    // P4

  ],
  "Max": [
    [7, 5, 3],
    // P0

    [3, 2, 2],
    // P1

    [9, 0, 2],
    // P2

    [2, 2, 2],
    // P3

    [4, 3, 3]
    // P4
  ]
}
```

```
]
}
```

1. Calculating the Need Matrix

The simulator first parses the configuration and derives the remaining resource requirement for each process using: $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$

```
user@os-sim:~/labs$ ./bankers --matrix need --file system_state.json
```

```
Processing system state...
```

```
Deriving NEED Matrix [Need = Max - Allocation]:
```

	Resource A	Resource B	Resource C
P0:	7	4	3
(7-0, 5-1, 3-0)			
P1:	1	2	2
(3-2, 2-0, 2-0)			
P2:	6	0	0
(9-3, 0-0, 2-2)			
P3:	0	1	1
(2-2, 2-1, 2-1)			
P4:	4	3	1
(4-0, 3-0, 3-2)			

```
Current Available Resources: [3, 3, 2]
```

2. Safety Algorithm Verification Trace

The algorithm executes by searching for a process whose remaining **Need** can be completely satisfied by the current **Available** pool. Once found, it simulates that process completing, releasing all its allocated resources back to the pool.

```
user@os-sim:~/labs$ ./bankers --check-safety --file system_state.json
Executing Safety Algorithm...

# Iteration 1

Checking P0: Need [7,4,3] <= Available [3,3,2] -> FALSE (Inverted resource
bottleneck)
Checking P1: Need [1,2,2] <= Available [3,3,2] -> TRUE
-> Executing Process P1...
-> P1 completes. Releasing Allocation [2,0,0]
-> New Available = [3,3,2] + [2,0,0] = [5,3,2]

# Iteration 2

Checking P0: Need [7,4,3] <= Available [5,3,2] -> FALSE
Checking P2: Need [6,0,0] <= Available [5,3,2] -> FALSE
Checking P3: Need [0,1,1] <= Available [5,3,2] -> TRUE
-> Executing Process P3...
-> P3 completes. Releasing Allocation [2,1,1]
-> New Available = [5,3,2] + [2,1,1] = [7,4,3]

# Iteration 3

Checking P4: Need [4,3,1] <= Available [7,4,3] -> TRUE
-> Executing Process P4...
-> P4 completes. Releasing Allocation [0,0,2]
-> New Available = [7,4,3] + [0,0,2] = [7,4,5]

# Iteration 4

Checking P0: Need [7,4,3] <= Available [7,4,5] -> TRUE
-> Executing Process P0...
-> P0 completes. Releasing Allocation [0,1,0]
-> New Available = [7,4,5] + [0,1,0] = [7,5,5]

# Iteration 5

Checking P2: Need [6,0,0] <= Available [7,5,5] -> TRUE
-> Executing Process P2...
-> P2 completes. Releasing Allocation [3,0,2]
-> New Available = [7,5,5] + [3,0,2] = [10,5,7] (All system resources
```

```
reclaimed)
```

RESULT: System is in a SAFE STATE.

Safe Sequence: < P1, P3, P4, P0, P2 >

3. Resource-Request Algorithm Simulation

When a process requests resources, the OS evaluates if granting them leaves the system in a safe state. Here, we simulate **P1 making a request of [1, 0, 2]**.

```
user@os-sim:~/labs$ ./bankers --request P1="1,0,2" --file system_state.json
```

```
Evaluating Resource Request from P1: [1, 0, 2]
```

```
Step 1: Is Request [1,0,2] <= P1_Need [1,2,2]? -> TRUE
```

```
Step 2: Is Request [1,0,2] <= Available [3,3,2]? -> TRUE
```

```
Step 3: Tentatively modifying tracking matrices...
```

```
Available = [3,3,2] - [1,0,2] = [2,3,0]
```

```
P1_Alloc = [2,0,0] + [1,0,2] = [3,0,2]
```

```
P1_Need = [1,2,2] - [1,0,2] = [0,2,0]
```

```
Running safety check on tentative state...
```

```
Checking sequence possibilities with new Available [2,3,0]...
```

```
P1 can run (Need [0,2,0] <= Available [2,3,0]).
```

```
P1 completes -> Available becomes [2,3,0] + [3,0,2] = [5,3,2]
```

```
From here, P3, P4, P0, P2 can all finish sequentially.
```

STATUS: REQUEST GRANTED.

The transaction is safe. Safe sequence remains valid: < P1, P3, P4, P0, P2 >